

编译原理

第九讲：语义分析

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 5 月 7 日



福州大学
FUZHOU UNIVERSITY

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

为什么需要修正语法树?

背景

为了适应 LL(1) 分析，我们通常需要消除左递归、提取左公因子。这些变换保持了语言的语法，却破坏了运算符原有的层级结构。

为什么需要修正语法树?

背景

为了适应 LL(1) 分析，我们通常需要消除左递归、提取左公因子。这些变换保持了语言的语法，却破坏了运算符原有的层级结构。

示例：左递归消除

原文法： $E \rightarrow E + T \mid T$

变换后：
 $E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \varepsilon$

为什么需要修正语法树?

背景

为了适应 LL(1) 分析，我们通常需要消除左递归、提取左公因子。这些变换保持了语言的语法，却破坏了运算符原有的层级结构。

示例：左递归消除

原文法： $E \rightarrow E + T \mid T$

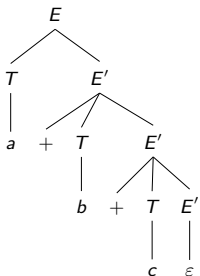
变换后：
 $E \rightarrow T E'$
 $E' \rightarrow + T E' \mid \varepsilon$

问题

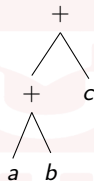
按变换后文法直接生成的语法树会把加法运算组织成右深链，不再是自然的二叉树结构，不便于后续语义处理。

原始语法树 vs 修正后的 AST

解析树 (直接生成)



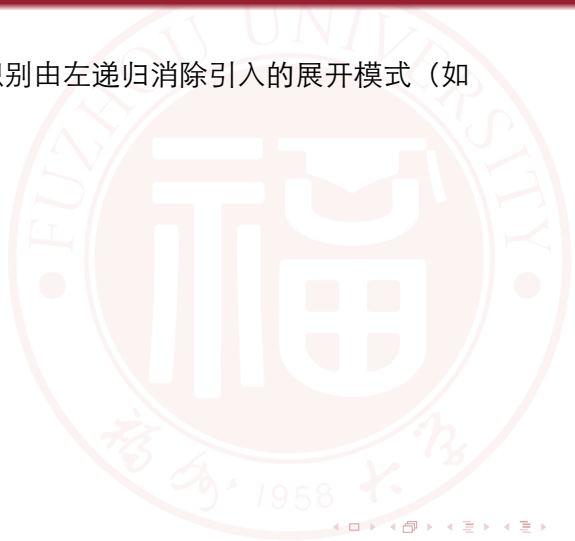
期望的 AST



- 直接生成的树带有大量 E' 节点，层次不分明；
- 需要将其折叠为以运算符为根、操作数为子树的 AST。

语法树修正的基本流程

- 1 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。



语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。
 - 左结合：从左向右依次折叠；

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。
 - 左结合：从左向右依次折叠；
 - 右结合：从右向左依次折叠。

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。
 - 左结合：从左向右依次折叠；
 - 右结合：从右向左依次折叠。
- ④ 标注属性：在新节点上标注类型、源位置等语义属性。

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。
 - 左结合：从左向右依次折叠；
 - 右结合：从右向左依次折叠。
- ④ 标注属性：在新节点上标注类型、源位置等语义属性。

语法树修正的基本流程

- ① 识别模式：在遍历解析树时，识别由左递归消除引入的展开模式（如 $E' \rightarrow +TE'$ ）。
- ② 收集操作数：将同一优先级上的连续运算收集到一个列表中（如 a, b, c ）。
- ③ 重建层级：按照运算符的结合性（左结合/右结合）重新构建二叉树。
 - 左结合：从左向右依次折叠；
 - 右结合：从右向左依次折叠。
- ④ 标注属性：在新节点上标注类型、源位置等语义属性。

实现策略

通常在语法分析的同时通过语义动作完成修正，或在分析完成后单独做一次树变换（**Tree Rewriting**）。

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;
- 类别 (Kind): 变量、函数、类型、常量等;

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;
- 类别 (Kind): 变量、函数、类型、常量等;
- 类型 (Type): 整型、浮点型、数组、指针、结构体等;

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;
- 类别 (Kind): 变量、函数、类型、常量等;
- 类型 (Type): 整型、浮点型、数组、指针、结构体等;
- 作用域 (Scope): 标识符的有效范围;

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;
- 类别 (Kind): 变量、函数、类型、常量等;
- 类型 (Type): 整型、浮点型、数组、指针、结构体等;
- 作用域 (Scope): 标识符的有效范围;
- 存储信息 (Address/Offset): 相对于栈帧或全局数据区的偏移;

符号表的作用

定义

符号表 (Symbol Table) 是编译器在语义分析阶段维护的核心数据结构，用于记录源程序中各类标识符的属性信息。

- 名字 (Name): 标识符的字符串表示;
- 类别 (Kind): 变量、函数、类型、常量等;
- 类型 (Type): 整型、浮点型、数组、指针、结构体等;
- 作用域 (Scope): 标识符的有效范围;
- 存储信息 (Address/Offset): 相对于栈帧或全局数据区的偏移;
- 其他属性: 是否初始化、访问控制、链接属性等。

符号表项的结构

域 (Field)	说明
Name	标识符名称
Kind	var / func / type / const
Type	指向类型描述结构的指针
Scope Level	嵌套深度 (全局为 0)
Offset	在栈帧或数据段中的偏移
Init?	是否已初始化
Linkage	链接属性 (内部/外部)

表 1: 典型的符号表项结构

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。



强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；
- 同一作用域内不允许多个同名强符号；

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；
- 同一作用域内不允许多个同名强符号；
- 强、弱符号同名时以强符号为准；

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；
- 同一作用域内不允许多个同名强符号；
- 强、弱符号同名时以强符号为准；
- 多个弱符号同名时由链接器选择其一。

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；
- 同一作用域内不允许多个同名强符号；
- 强、弱符号同名时以强符号为准；
- 多个弱符号同名时由链接器选择其一。

强弱符号处理

概念

在 C/C++ 等语言的编译与链接模型中，符号按定义方式分为强符号（Strong）与弱符号（Weak）。

- 强符号：已初始化的全局变量或函数定义；
- 弱符号：未初始化的全局变量（如 `int x;`）；
- 同一作用域内不允许多个同名强符号；
- 强、弱符号同名时以强符号为准；
- 多个弱符号同名时由链接器选择其一。

语义分析阶段任务

记录符号的链接属性，为后续链接提供依据。

作用域与多层符号表

作用域类型

- 全局作用域
- 函数/过程作用域
- 块作用域 ({})
- 类/结构体作用域

实现方式

- 栈式符号表 (Scope Stack)
- 哈希表 + 链表 (处理冲突与嵌套)
- 树形符号表 (每个节点代表一层作用域)

作用域与多层符号表

作用域类型

- 全局作用域
- 函数/过程作用域
- 块作用域 ({})
- 类/结构体作用域

实现方式

- 栈式符号表 (Scope Stack)
- 哈希表 + 链表 (处理冲突与嵌套)
- 树形符号表 (每个节点代表一层作用域)

查找规则

从内层作用域逐层向外查找 (最近嵌套规则), 确保局部变量遮蔽 (Shadowing) 外层同名变量。

多层符号表示例

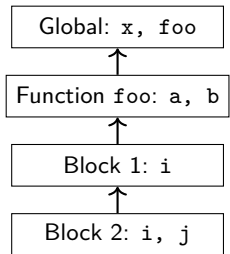


图 1: 作用域链 (Scope Chain)

- 进入新块时压入新层，退出时弹出；

多层符号表示例

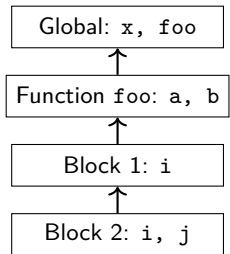


图 1: 作用域链 (Scope Chain)

- 进入新块时压入新层，退出时弹出；
- 同一层内不允许重复声明；

多层符号表示例

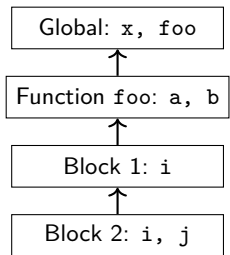


图 1: 作用域链 (Scope Chain)

- 进入新块时压入新层，退出时弹出；
- 同一层内不允许重复声明；
- 外层符号对内层可见，除非被遮蔽。

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

为什么需要隐式转换?

示例

```
int a = 3;  
double b = a + 2.5;
```

为什么需要隐式转换?

示例

```
int a = 3;
```

```
double b = a + 2.5;
```

- 表达式 $a + 2.5$ 中, a 为 `int`, 2.5 为 `double`;

为什么需要隐式转换?

示例

```
int a = 3;
```

```
double b = a + 2.5;
```

- 表达式 $a + 2.5$ 中, a 为 `int`, 2.5 为 `double`;
- 多数语言规定: 混合类型运算时, 将较窄类型提升为较宽类型;

为什么需要隐式转换?

示例

```
int a = 3;  
double b = a + 2.5;
```

- 表达式 $a + 2.5$ 中, a 为 `int`, 2.5 为 `double`;
- 多数语言规定: 混合类型运算时, 将较窄类型提升为较宽类型;
- 语义分析器需在 AST 中显式插入类型转换节点 (**Cast Node**)。

类型提升规则与 AST 修正

常见算术转换规则

操作数 1	操作数 2	结果
int	int	int
int	float	float
int	double	double
float	double	double

AST 修正示意



要点

在 AST 中相应位置插入 Cast 节点，标注源类型与目标类型。

隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；



隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；
- ② 类型检查：判断运算是否合法，若不匹配则查找转换规则；

隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；
- ② 类型检查：判断运算是否合法，若不匹配则查找转换规则；
- ③ 插入转换节点：在 AST 中插入 Cast 节点；

隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；
- ② 类型检查：判断运算是否合法，若不匹配则查找转换规则；
- ③ 插入转换节点：在 AST 中插入 Cast 节点；
- ④ 报错或警告：若不存在合法的隐式转换路径，则报告类型错误。

隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；
- ② 类型检查：判断运算是否合法，若不匹配则查找转换规则；
- ③ 插入转换节点：在 AST 中插入 Cast 节点；
- ④ 报错或警告：若不存在合法的隐式转换路径，则报告类型错误。

隐式转换的处理流程

- ① 类型推导：自底向上遍历 AST，计算每个表达式的结果类型；
- ② 类型检查：判断运算是否合法，若不匹配则查找转换规则；
- ③ 插入转换节点：在 AST 中插入 Cast 节点；
- ④ 报错或警告：若不存在合法的隐式转换路径，则报告类型错误。

注意

隐式转换可能导致精度丢失（如 double 转 float），编译器通常应给出警告。

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

什么是语法糖？

定义

语法糖 (Syntactic Sugar) 是语言提供的便捷记法，它不改变语言的表达能力，但能让代码更简洁、更易读。

常见语法糖示例

- 复合赋值: $a += b \Leftrightarrow a = a + b$
- 自增/自减: $a++ \Leftrightarrow a = a + 1$
- 条件表达式: $a ? b : c \Leftrightarrow \text{if-else 赋值形式}$
- 范围 for: $\text{for } (x : \text{list}) \Leftrightarrow \text{迭代器循环}$
- 下标访问: $a[i] \Leftrightarrow *(a + i)$

去糖化 (Desugaring) 过程

核心思想

在语义分析阶段，将语法糖节点翻译为语言核心构造的等价形式，简化后续代码生成。

去糖化 (Desugaring) 过程

核心思想

在语义分析阶段，将语法糖节点翻译为语言核心构造的等价形式，简化后续代码生成。

示例：复合赋值

`x += expr`

↓ **Desugar**

`x = x + (expr)`

注意副作用

去糖化时必须保证求值顺序一致。例如 `a[f()] += b` 不应直接去糖化为 `a[f()] = a[f()] + b`，而应引入临时变量保存左值地址。

语法糖处理流程

- 1 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；



语法糖处理流程

- ① 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；
- ② 展开为核心节点：按语言规范重写为等价的核心语法树；

语法糖处理流程

- ① 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；
- ② 展开为核心节点：按语言规范重写为等价的核心语法树；
- ③ 处理边界情况：

语法糖处理流程

- ① 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；
- ② 展开为核心节点：按语言规范重写为等价的核心语法树；
- ③ 处理边界情况：
 - 保证副作用只执行一次；

语法糖处理流程

- ① 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；
- ② 展开为核心节点：按语言规范重写为等价的核心语法树；
- ③ 处理边界情况：
 - 保证副作用只执行一次；
 - 保留类型检查和隐式转换；

语法糖处理流程

- ① 识别糖节点：在 AST 中标记属于语法糖的节点类型（如 CompoundAssign、PostInc）；
- ② 展开为核心节点：按语言规范重写为等价的核心语法树；
- ③ 处理边界情况：
 - 保证副作用只执行一次；
 - 保留类型检查和隐式转换；
- ④ 递归去糖：展开后的子树中可能仍包含语法糖，需递归处理直到完全消除。

目录

① 语法树的修正

② 符号表的构建与管理

③ 隐式转换的添加

④ 语法糖的翻译与处理

⑤ 基于 ANTLR 的语法检查示例

ANTLR 简介

ANTLR

ANTLR (ANother Tool for Language Recognition) 是一个强大的解析器生成器，支持 LL(*) 文法，可自动生成词法分析器、语法分析器以及 **Visitor/Listener** 框架。

ANTLR 简介

ANTLR

ANTLR (ANother Tool for Language Recognition) 是一个强大的解析器生成器，支持 LL(*) 文法，可自动生成词法分析器、语法分析器以及 **Visitor/Listener** 框架。

Visitor 模式

主动遍历语法树，适合需要返回值的语义分析。

Listener 模式

通过回调响应进入/退出节点事件，适合代码生成。

表达式文法定义 (ANTLR 4)

```
1 grammar Expr;
2 prog : stat+ ;
3 stat : expr ';' | ID '=' expr ';' ;
4 expr : expr ('+' | '-') expr # AddSub
5       | expr ('*' | '/') expr # MulDiv
6       | INT                  # Int
7       | ID                   # Id
8       | '(' expr ')'         # Parens
9       ;
10 INT : [0-9]+ ;
11 ID  : [a-zA-Z]+ ;
12 WS  : [ \t\r\n]+ -> skip ;
```

Visitor 模式实现类型检查

```
1 public class TypeChecker extends ExprBaseVisitor<Type> {  
2     SymbolTable symTable = new SymbolTable();  
3     @Override  
4     public Type visitAddSub(ExprParser.AddSubContext ctx) {  
5         Type left = visit(ctx.expr(0));  
6         Type right = visit(ctx.expr(1));  
7         if (!left.equals(right))  
8             return promote(left, right); // 隐式转换或报错  
9         return left;  
10    }  
11    @Override  
12    public Type visitId(ExprParser.IdContext ctx) {  
13        return symTable.lookup(ctx.ID().getText()).type;  
14    }  
15 }
```

ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；



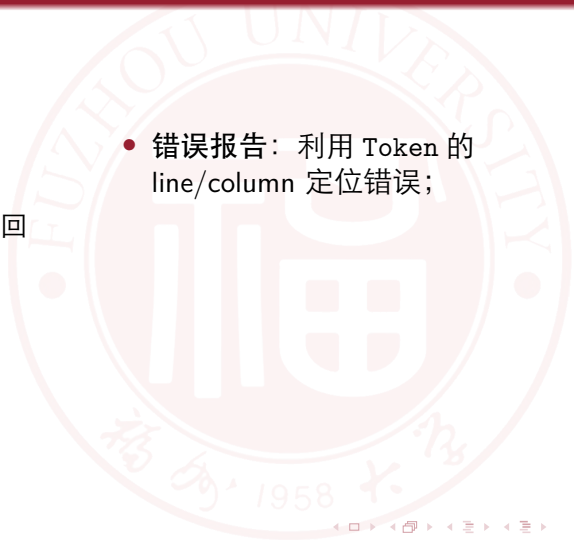
ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；
- 类型推导：每个 `visitXxx` 方法返回子树的类型；



ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；
- 类型推导：每个 `visitXxx` 方法返回子树的类型；
- 错误报告：利用 `Token` 的 `line/column` 定位错误；



ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；
- 类型推导：每个 `visitXxx` 方法返回子树的类型；
- 错误报告：利用 `Token` 的 `line/column` 定位错误；
- **AST 转换**：在 `Visitor` 中直接构造修正后的 `AST` 节点。

ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；
- 类型推导：每个 `visitXxx` 方法返回子树的类型；
- 错误报告：利用 `Token` 的 `line/column` 定位错误；
- **AST 转换**：在 `Visitor` 中直接构造修正后的 `AST` 节点。

ANTLR 语义检查要点

- 符号表维护：在 `visitProg` 或 `enterStat` 时建立作用域；
- 类型推导：每个 `visitXxx` 方法返回子树的类型；
- 错误报告：利用 `Token` 的 `line/column` 定位错误；
- **AST 转换**：在 `Visitor` 中直接构造修正后的 `AST` 节点。

优势

ANTLR 自动生成的树遍历框架使得语义分析代码与文法解耦，便于维护和扩展。

本讲小结

① 语法树修正

LL(1) 变换后的解析树需还原为语义清晰的 AST，核心是按结合性折叠链式结构。

② 符号表管理

符号表记录名字、类型、作用域、链接属性等信息；多层符号表通过栈或链式结构实现作用域嵌套与查找。

③ 隐式转换

混合类型运算时需在 AST 中插入转换节点，并检查是否合法。

④ 语法糖去糖化

将便捷记法展开为核心语法构造，注意副作用与求值顺序。

⑤ ANTLR 实践

利用 Visitor/Listener 模式可高效实现类型检查、符号表管理与树变换。